

Web Application Development

Lecture 10: Node.js, npm & Async JS

SWE 381 – Web Application Development

Dr. Ohoud Alharbi Dr. Achraf Gazdar

Department of Software Engineering
College of Computer and Information Sciences
King Saud University

Term 2 – 2025/2026

Learning Objectives

By the end of this lecture, you will be able to:

- Explain the key differences between **Browser JavaScript** and **Node.js**
- Describe the **event loop** and **non-blocking I/O** model
- Use **CommonJS modules** – `require()` and `module.exports`
- Initialise a Node.js project with `npm init` and manage packages
- Write and run **npm scripts**, including `start` and `dev`
- Set up **nodemon** for automatic server restarts during development
- Load secret configuration using **dotenv** and `.env` files
- Organise a Node.js backend with a **professional folder structure**

Agenda

- 1 Browser JS vs Node.js
- 2 Event Loop & Non-Blocking I/O
- 3 CommonJS: require / module.exports
- 4 npm init, install, scripts
- 5 nodemon
- 6 dotenv & .env Secrets
- 7 Backend Folder Structure
- 8 Recap

Outline

- 1 Browser JS vs Node.js
- 2 Event Loop & Non-Blocking I/O
- 3 CommonJS: require / module.exports
- 4 npm init, install, scripts
- 5 nodemon
- 6 dotenv & .env Secrets
- 7 Backend Folder Structure
- 8 Recap

What is Node.js?

W3Schools – Node.js Introduction

Node.js is a **free, open-source JavaScript runtime** that runs on Windows, Mac, Linux, and more. It lets you execute JavaScript code **outside of a web browser**, enabling server-side development with JavaScript. Built on Chrome's **V8 JavaScript engine**, Node.js is designed for building scalable network applications efficiently.

```
// hello.js -- run with: node hello.js
console.log('Hello from Node.js!');

// Access the file system (no browser!)
const fs = require('fs');
fs.readFile('notes.txt', 'utf8', (err, data) => {
  if (err) { console.error(err); return; }
  console.log('File contents:', data);
});

// Simple HTTP server
const http = require('http');
http.createServer((req, res) => {
  res.end('Hello World!');
}).listen(3000);
console.log('Server on port 3000');
```

Node.js is perfect for:

- REST APIs and microservices
- Real-time chat applications
- Data streaming services
- Command-line tools
- Server-side web applications

Terminal: node hello.js

Hello from Node.js!

File contents: My notes

Server on port 3000

Browser JS vs Node.js – Key Differences

W3Schools – Node.js vs Browser

Node.js and browsers both run JavaScript, but they have different environments and capabilities. Node.js is designed for **server-side development**, while browsers are for **client-side applications**.

Feature	Browser JS	Node.js
Where it runs	Inside a browser	On the server / OS
DOM access	Yes (document, window)	No DOM
File system	No (sandboxed)	Yes (fs module)
Network access	fetch, XHR	http, https modules
Modules	ES Modules (import)	CommonJS (require) or ESM
Environment vars	No	Yes (process.env)
Package manager	CDN / bundler	npm / yarn
Global object	window	global / globalThis
Use case	UI, interactivity	Servers, APIs, tools

The Same JavaScript – Different Environments

Browser JavaScript:

```
// Global object: window
window.myVar = 42;
console.log(window.myVar); // 42

// Has DOM access
document.getElementById('btn')
  .addEventListener('click', () => {
    alert('Clicked!');
  });

// Has fetch API
fetch('/api/data')
  .then(res => res.json())
  .then(data => console.log(data));

// NO file system access
// fs.readFile('...') -- ERROR!
```

Node.js:

```
1 // Global object: global
2 global.myVar = 42;
3 console.log(global.myVar); // 42
4
5 // NO DOM access
6 // document.getElementById -- ERROR!
7
8 // Has file system access
9 const fs = require('fs');
10 fs.readFile('data.txt', 'utf8',
11   (err, data) => console.log(data)
12 );
13
14 // Has HTTP server capability
15 const http = require('http');
16 http.createServer((req, res) => {
17   res.end('Hello from Node!');
18 }).listen(8080);
```



Same language, different superpowers

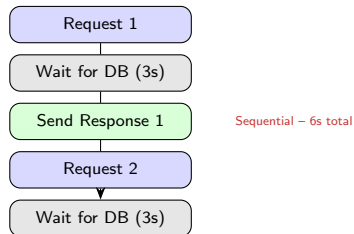
JavaScript is the same language in both environments. What changes is the **runtime environment** and the APIs available. Node.js gains file/network access; it loses the DOM.

Outline

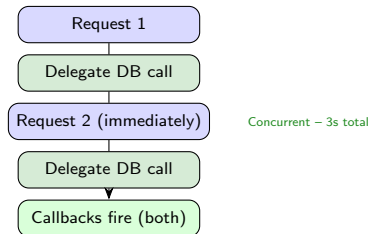
- 1 Browser JS vs Node.js
- 2 Event Loop & Non-Blocking I/O
- 3 CommonJS: require / module.exports
- 4 npm init, install, scripts
- 5 nodemon
- 6 dotenv & .env Secrets
- 7 Backend Folder Structure
- 8 Recap

Blocking vs Non-Blocking I/O

Blocking (traditional server):



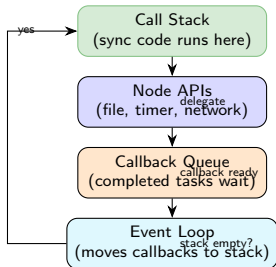
Non-Blocking (Node.js):



The Event Loop Explained

W3Schools – Node.js Event Loop

The event loop makes Node.js **non-blocking**: it delegates async tasks to the system and processes results via callbacks, handling **thousands of connections** on a single thread.



```
1  const fs = require('fs');
2  console.log('Before file read');
3  fs.readFile('myfile.txt', 'utf8', (err, data) => {
4    if (err) throw err;
5    console.log('File contents:', data);
6  });
7  console.log('After file read');
```

Output

Before file read / After file read

File contents: Hello World

“After file read” prints BEFORE file contents.

Sync vs Async Code – Side by Side

Synchronous (blocking):

```
1 const fs = require('fs');  
2 // readFileSync BLOCKS until file is read  
3 const data = fs.readFileSync('courses.txt', 'utf8');  
4 console.log(data);  
5 console.log('This waits...');
```

Output (order)

[file contents] / This waits...

Asynchronous (non-blocking):

```
1 const fs = require('fs');  
2 // readFile does NOT block  
3 fs.readFile('courses.txt', 'utf8',  
4   (err, data) => {  
5     if (err) throw err;  
6     console.log(data);  
7   }  
8 );  
9 console.log('Free to do other work!');
```

Output (order)

Free to do other work! / [file contents]

Outline

- 1 Browser JS vs Node.js
- 2 Event Loop & Non-Blocking I/O
- 3 CommonJS: require / module.exports
- 4 npm init, install, scripts
- 5 nodemon
- 6 dotenv & .env Secrets
- 7 Backend Folder Structure
- 8 Recap

CommonJS Modules – Overview

W3Schools – Node.js Modules

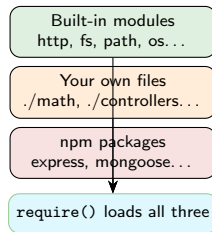
In Node.js, any file with a **.js extension is a module**. You can export functionality from a module in several ways, and use `require()` to load it in other files.

Three types of modules:

```
// 1. Built-in (no install needed)
const http = require('http');
const fs   = require('fs');
const path = require('path');
const os   = require('os');

// 2. Your own files (relative path)
const math      = require('./math');
const userCtrl  = require('./controllers/users');

// 3. Installed packages (from npm)
const express   = require('express');
const mongoose  = require('mongoose');
const dotenv    = require('dotenv');
```



module.exports – Exporting from a Module

```
// ---- math.js ---- export multiple functions via an object
function add(a, b)      { return a + b; }
function subtract(a, b) { return a - b; }
function multiply(a, b) { return a * b; }

// Export all three as named properties
module.exports = { add, subtract, multiply };

// ---- OR: export a single thing directly ----
// module.exports = add; // exports just the add function

// ---- greet.js ---- export an object with properties
const PREFIX = 'Hello';
module.exports = {
  greet: (name) => `${PREFIX}, ${name}!`,
  farewell: (name) => `Goodbye, ${name}.`
};

// ---- UserModel.js ---- export a class
class User {
  constructor(name, email) {
    this.name = name;
    this.email = email;
  }
  toString() { return `${this.name} <${this.email}>`; }
}
module.exports = User;
```

require() – Importing a Module

```
// 1. Named functions
const { add, subtract } = require('./math');
console.log(add(5, 3)); // 8
console.log(subtract(10, 4)); // 6
// 2. Whole exported object
const math = require('./math');
console.log(math.multiply(3, 4)); // 12
// 3. Functions from greet module
const { greet, farewell } = require('./greet');
console.log(greet('Ahmed')); // Hello, Ahmed!
console.log(farewell('Sara')); // Goodbye, Sara.
// 4. Class import
const User = require('./UserModel');
const u = new User('Khalid', 'k@ksu.edu.sa');
console.log(u.toString()); // Khalid <k@ksu.edu.sa>
// 5. Built-in module
const os = require('os');
console.log('OS:', os.platform(), os.cpus().length);
```

Output

8 / 6 / 12

Hello, Ahmed!

Goodbye, Sara.

Khalid <k@ksu.edu.sa>

OS: linux 4

Path	Meaning
'./math'	same folder
'../lib'	parent
'express'	npm pkg
'fs'	built-in

Caching

require() **caches** on first load – singleton.

CommonJS vs ES Modules

Feature	CommonJS (CJS)	ES Modules (ESM)
Syntax	<code>require()</code> / <code>module.exports</code>	<code>import</code> / <code>export</code>
File extension	<code>.js</code> (default)	<code>.mjs</code> or <code>"type": "module"</code>
When loads	Runtime (dynamic)	Parse time (static)
Browser support	No (Node.js only)	Yes (modern browsers)
Used in	Node.js backends	React, modern Node.js
Async capable	No	Yes (top-level <code>await</code>)

```
1 // CommonJS (backend)
2 const express = require('express');
3 module.exports = { router };
```

```
1 // ES Modules (React frontend)
2 import React from 'react';
3 export default function App() {}
```



This course uses CommonJS for the backend

We use `require` and `module.exports` for our Express backend. React frontend uses ES Modules (`import/export`) as you learned in lectures 1–9.

Outline

- 1 Browser JS vs Node.js
- 2 Event Loop & Non-Blocking I/O
- 3 CommonJS: require / module.exports
- 4 **npm init, install, scripts**
- 5 nodemon
- 6 dotenv & .env Secrets
- 7 Backend Folder Structure
- 8 Recap

npm – The Node Package Manager

W3Schools – npm

npm is the **Node Package Manager** – the world's largest software registry with over **1.3 million packages**. Auto-installed with Node.js; lets you install, share, and manage packages.

Most important npm commands:

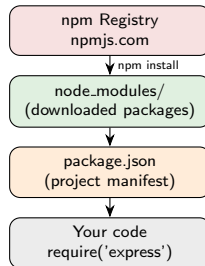
```
node --version    # v20.x.x
npm --version     # 10.x.x

npm init          # interactive wizard
npm init -y       # skip all questions

npm install express      # prod dep
npm install nodemon --save-dev # dev dep
npm install              # restore all

npm uninstall express

npm start
npm run dev
npm test
```



npm init – Creating a Project

Running `npm init -y`:

```
$ mkdir swe381-backend && cd swe381-backend
$ npm init -y
```

Generated `package.json`:

```
{
  "name": "swe381-backend",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": { "test": "echo no test" },
  "license": "ISC"
}
```

`package.json` key fields:

- `name` – project name
- `version` – MAJOR.MINOR.PATCH
- `main` – entry point file
- `scripts` – runnable commands
- `dependencies` – runtime packages
- `devDependencies` – dev-only packages

W3Schools – `package.json`

Describes the app, sets entry to `index.js`, and defines scripts. Installed packages are added to **dependencies** automatically.

npm install – Adding Packages

```
# Install Express (production)
$ npm install express

# Install dev tools (dev only)
$ npm install nodemon --save-dev
$ npm install dotenv --save-dev

# Shorthand: -D means --save-dev
$ npm i nodemon dotenv -D
```

Updated package.json:

```
{
  "name": "swe381-backend",
  "version": "1.0.0",
  "main": "index.js",
  "dependencies": {
    "express": "^4.18.2"
  },
  "devDependencies": {
    "nodemon": "^3.0.1",
    "dotenv": "^16.3.1"
  }
}
```

Difference:

dependencies	devDependencies
Used in production express, mongoose npm install pkg	Development only nodemon, dotenv npm install pkg -D



.gitignore node_modules

Never commit node_modules/ to Git – it can contain thousands of files. Add it to .gitignore. Anyone can restore all packages by running npm install.



Version prefix ^

^4.18.2 means “compatible with 4.x.x”. npm updates to the latest minor/patch but not major version.

npm scripts – Automate Your Workflow

W3Schools – npm Scripts

Define custom scripts that can be run with `npm run <script-name>`. The start and test scripts are special – they can be run without the run keyword.

package.json scripts section:

```
{
  "scripts": {
    "start": "node index.js",
    "dev": "nodemon index.js",
    "test": "jest",
    "build": "webpack --mode production",
    "lint": "eslint ."
  }
}
```

Running scripts:

```
npm start      # runs: node index.js
npm run dev    # runs: nodemon index.js
npm test       # runs: jest
npm run build  # runs: webpack ...
npm run lint   # runs: eslint .
```

Common script patterns:

- "start": production server
- "dev": development with auto-restart
- "test": run test suite
- "build": compile/bundle
- "lint": check code style



Why use scripts?

Scripts let you **standardise** commands across your team. Instead of remembering the exact nodemon command, everyone runs `npm run dev`. New developers run `npm start` without knowing the internals.

Outline

- 1 Browser JS vs Node.js
- 2 Event Loop & Non-Blocking I/O
- 3 CommonJS: require / module.exports
- 4 npm init, install, scripts
- 5 nodemon**
- 6 dotenv & .env Secrets
- 7 Backend Folder Structure
- 8 Recap

nodemon – Auto-Restart on Save

nodemon

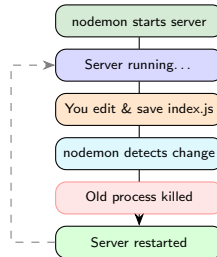
nodemon automatically restarts your Node.js app on file changes – no manual server restarts needed.

Install and use:

```
$ npm install nodemon --save-dev  
$ npx nodemon index.js # or: npm run dev
```

package.json:

```
{  
  "scripts": {  
    "start": "node index.js",  
    "dev": "nodemon index.js"  
  },  
  "devDependencies": { "nodemon": "~3.0.1" }  
}
```



nodemon – Terminal Output

```
$ npm run dev
> swe381-backend@1.0.0 dev
> nodemon index.js

[nodemon] 3.0.1
[nodemon] to restart at any time,
  enter 'rs'
[nodemon] watching path(s): *.*
[nodemon] watching extensions:
  js,mjs,cjs,json
[nodemon] starting 'node index.js'

Server running on port 3000

[nodemon] restarting due to changes...
[nodemon] starting 'node index.js'

Server running on port 3000
```

Without nodemon:

- 1 Edit a file
- 2 Press Ctrl+C to kill server
- 3 Type `node index.js` again
- 4 Repeat on every change

With nodemon:

- 1 Edit a file and save
- 2 nodemon auto-restarts



Development only

nodemon is a **dev dependency** – it's not needed in production. In production, you use `npm start` which runs `node index.js` directly.

Outline

- 1 Browser JS vs Node.js
- 2 Event Loop & Non-Blocking I/O
- 3 CommonJS: require / module.exports
- 4 npm init, install, scripts
- 5 nodemon
- 6 dotenv & .env Secrets**
- 7 Backend Folder Structure
- 8 Recap

Why Environment Variables?

The problem – hardcoded secrets:

```
// WRONG -- secrets in source code!  
const mongoose = require('mongoose');  
  
mongoose.connect(  
  'mongodb://admin:mypassword123@db.host/swe381'  
);  
  
const jwt = require('jsonwebtoken');  
const token = jwt.sign(  
  { id: user._id },  
  'mySuper$ecretKey12345' // exposed in git!  
);
```



Never commit secrets to Git

Pushing API keys, passwords, or tokens to a public repository is a **critical security risk**. Attackers scan GitHub for exposed credentials.

The solution – .env file:

```
# .env (never commit this file!)  
PORT=3000  
MONGO_URI=mongodb://admin:pass@db/swe381  
JWT_SECRET=mySuper$ecretKey12345  
NODE_ENV=development  
DB_HOST=localhost  
DB_USER=admin  
DB_PASS=password
```

```
# .gitignore  
node_modules/  
.env  
.env.local  
dist/
```

dotenv – Loading .env into process.env

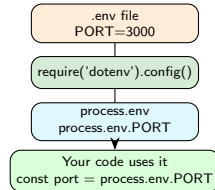
W3Schools – dotenv

The **dotenv** package loads a **.env** file into **process.env**. Never commit **.env** to version control.

Setup and usage:

```
// npm install dotenv --save-dev
// Load at TOP of index.js
require('dotenv').config();

// Access via process.env
const port = process.env.PORT || 3000;
const mongo = process.env.MONGO_URI;
const jwt = process.env.JWT_SECRET;
const env = process.env.NODE_ENV;
const dbConfig = {
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  pass: process.env.DB_PASS
};
console.log(`Port: ${port}, Env: ${env}`);
// Port: 3000, Env: development
```



💡 Fallback with ||

Use `process.env.PORT || 3000` to provide a **default value** when the variable is not set.

.env Best Practices

A complete .env file:

```
PORT=3000
NODE_ENV=development
MONGO_URI=mongodb://localhost:27017/swe381
DB_HOST=localhost
DB_USER=admin
DB_PASS=secret123
JWT_SECRET=swe381-very-long-random-secret
JWT_EXPIRE=7d
EMAIL_API_KEY=sg.abc123...
STRIPE_KEY=sk_test_abc123...
```

.env.example (safe to commit):

```
PORT=3000
NODE_ENV=development
MONGO_URI=          # Add your MongoDB URI
JWT_SECRET=         # Add a random secret
EMAIL_API_KEY=      # Add your SendGrid key
```

Rule	Why
UPPER_CASE names	Convention
No spaces around =	Syntax
No quotes needed	dotenv handles it
Add to .gitignore	Security
Commit .env.example	Onboarding



require at the very top

Call `require('dotenv').config()` as the **first line** of your entry file (`index.js`).



Dev vs Production

In production (Heroku, Vercel, AWS), set variables through the **hosting platform dashboard**.

Outline

- 1 Browser JS vs Node.js
- 2 Event Loop & Non-Blocking I/O
- 3 CommonJS: require / module.exports
- 4 npm init, install, scripts
- 5 nodemon
- 6 dotenv & .env Secrets
- 7 Backend Folder Structure**
- 8 Recap

Professional Backend Folder Structure

```
swe381-backend/  
src/  
  controllers/    <- request handlers  
    userCtrl.js  
    courseCtrl.js  
  routes/        <- URL definitions  
    userRoutes.js  
    courseRoutes.js  
  models/         <- DB schemas  
    User.js  
    Course.js  
  middleware/     <- reusable fns  
    authMiddleware.js  
    errorHandler.js  
  config/         <- configuration  
    db.js  
    email.js  
  utils/          <- helper fns  
    generateToken.js  
    sendEmail.js  
  index.js        <- entry point  
package.json  
.env              <- secrets (gitignore!)  
.env.example      <- template (safe)  
.gitignore
```

Each folder's responsibility:

- **controllers/** – Business logic: handle requests, call models, send responses
- **routes/** – Define URL paths and map them to controllers
- **models/** – Database schemas and data access
- **middleware/** – Functions between request and response (auth, logging, error handling)
- **config/** – Database connection, external service setup
- **utils/** – Reusable helper functions
- **index.js** – App entry point: loads middleware, routes, starts server

Backend Structure – index.js

```
// index.js -- entry point -- FIRST thing: load dotenv
require('dotenv').config();

const express = require('express');
const app      = express();

// ---- Middleware ----
app.use(express.json()); // parse JSON request bodies

// ---- Routes ----
const userRoutes   = require('./src/routes/userRoutes');
const courseRoutes = require('./src/routes/courseRoutes');

app.use('/api/users',   userRoutes);
app.use('/api/courses', courseRoutes);

// ---- Root health check ----
app.get('/', (req, res) => {
  res.json({ message: 'SWE381 API is running!' });
});

// ---- Start server ----
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log('Server running on port ${PORT}');
  console.log('Environment: ${process.env.NODE_ENV}');
});
```

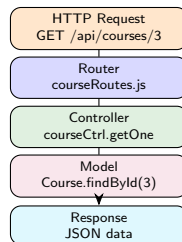
Backend Structure – Routes and Controllers

courseRoutes.js:

```
const router      = require('express').Router();
const courseCtrl = require('../controllers/courseCtrl');
router.get('/',      courseCtrl.getAll);
router.get('/:id',   courseCtrl.getOne);
router.post('/',      courseCtrl.create);
router.put('/:id',    courseCtrl.update);
router.delete('/:id', courseCtrl.remove);
module.exports = router;
```

courseCtrl.js:

```
const getAll = (req, res) => {
  res.json({ courses: [] });
};
const getOne = (req, res) => {
  const { id } = req.params;
  res.json({ course: { id, title: 'SWE 381' } });
};
module.exports = { getAll, getOne };
```



Quick-Start: New Backend Project

```
# 1. Create project folder
mkdir swe381-backend && cd swe381-backend

# 2. Initialise npm project
npm init -y

# 3. Install production dependencies
npm install express

# 4. Install dev dependencies
npm install nodemon dotenv --save-dev

# 5. Create files
touch index.js .env .gitignore .env.example

# 6. Add to .gitignore
echo "node_modules/" >> .gitignore
echo ".env" >> .gitignore

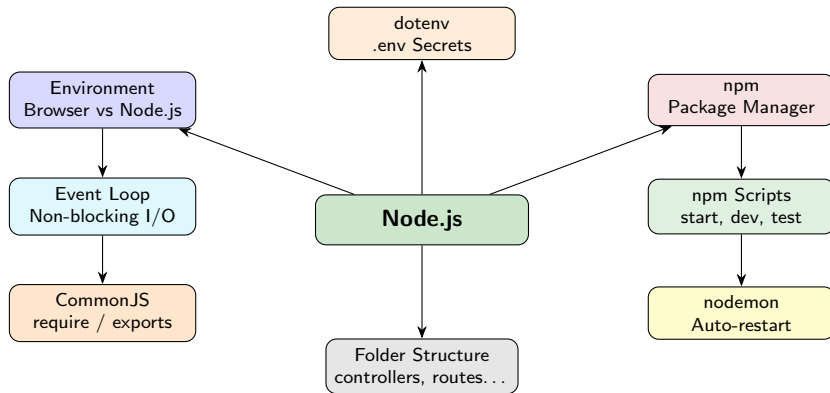
# 7. Update package.json scripts
#   "start": "node index.js",
#   "dev": "nodemon index.js"

# 8. Run in development
npm run dev
```

Outline

- 1 Browser JS vs Node.js
- 2 Event Loop & Non-Blocking I/O
- 3 CommonJS: require / module.exports
- 4 npm init, install, scripts
- 5 nodemon
- 6 dotenv & .env Secrets
- 7 Backend Folder Structure
- 8 Recap

Node.js & npm – Concept Map



Lecture 10 – Key Takeaways

- 1 **Node.js** runs JavaScript on the server – same language as React, but different APIs (file system, http, no DOM)
- 2 Browser JS has `window`, `DOM`, `fetch`; Node.js has `global`, `fs`, `http`, `process.env`
- 3 The **event loop** makes Node.js non-blocking – I/O tasks are delegated to the system; the main thread never waits
- 4 **Non-blocking code** (callback, `async/await`) is essential in Node.js servers – blocking code freezes all requests
- 5 **CommonJS**: `require()` to import, `module.exports` to export – default module system for Node.js backends
- 6 `npm init` creates `package.json`; `npm install` adds packages to dependencies or `devDependencies`
- 7 **npm scripts** standardise commands: `npm start` for production, `npm run dev` for development
- 8 **nodemon** watches files and auto-restarts the server on save – install as `devDependency` only
- 9 **dotenv** loads `.env` files into `process.env` – never commit `.env` to Git; use `.env.example` as a template
- 10 **Backend structure**: `controllers/`, `routes/`, `models/`, `middleware/`, `config/`, `utils/`, `index.js`

Next Lecture: Express.js – Building REST APIs with Middleware and Routes

Practice Exercises

- 1 Install Node.js. Create `hello.js` and run `console.log('Hello World')` with `node hello.js`. Then use the built-in `fs` module to read a text file asynchronously (callback style) and print its contents.
- 2 Demonstrate the event loop: in `order.js` log "Start", then schedule `setTimeout(() => console.log('Timeout'), 0)`, then log "End". Confirm that "End" prints before "Timeout".
- 3 Create a **CommonJS module** `mathUtils.js` that exports `add`, `subtract`, and `multiply`. Import all three functions in `app.js` using both **destructuring** and the **whole module object**. Print results.
- 4 Run `npm init -y` in a new folder. Install **nodemon** as a dev dependency. Add a "dev": `"nodemon index.js"` script to `package.json`. Start the server with `npm run dev` and confirm it restarts when you edit the file.
- 5 Create a `.env` file with `PORT`, `NODE_ENV`, and `DB_PASS`. Install **dotenv**. Load it in `index.js` and print all three values. Verify `.env` is listed in `.gitignore`.
- 6 Set up the full **backend folder structure** from this lecture (`src/controllers`, `src/routes`, etc.). Create a `courseRoutes.js` with 5 routes and a `courseCtrl.js` that returns mock JSON responses for each.
- 7 **Challenge:** Build a minimal Express app combining all concepts – `dotenv` for configuration, CommonJS modules for routes/controllers, `nodemon` for development, and the professional folder structure. The API should respond to `GET /api/courses` with a JSON array.